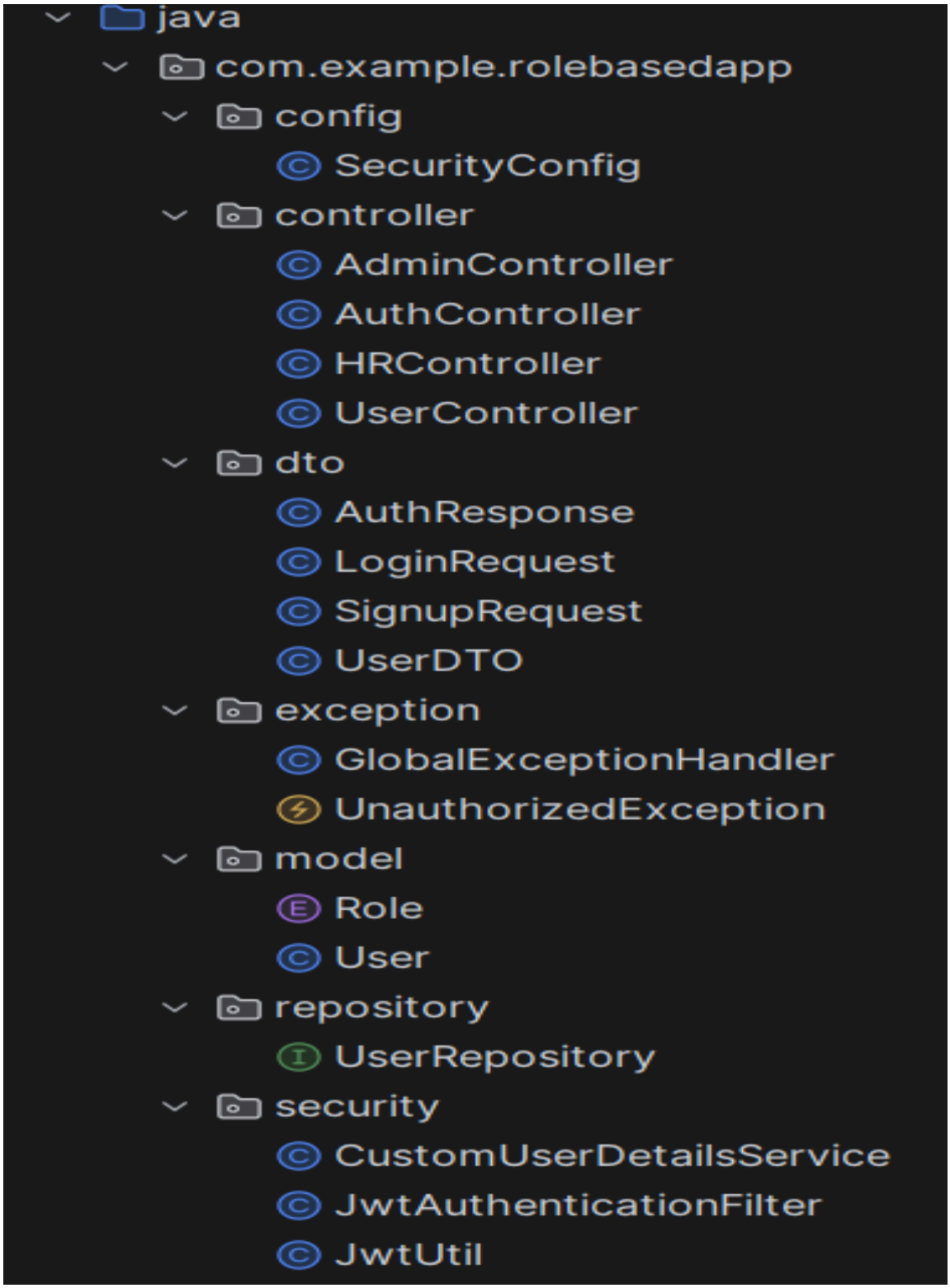
Role Based Routing Application

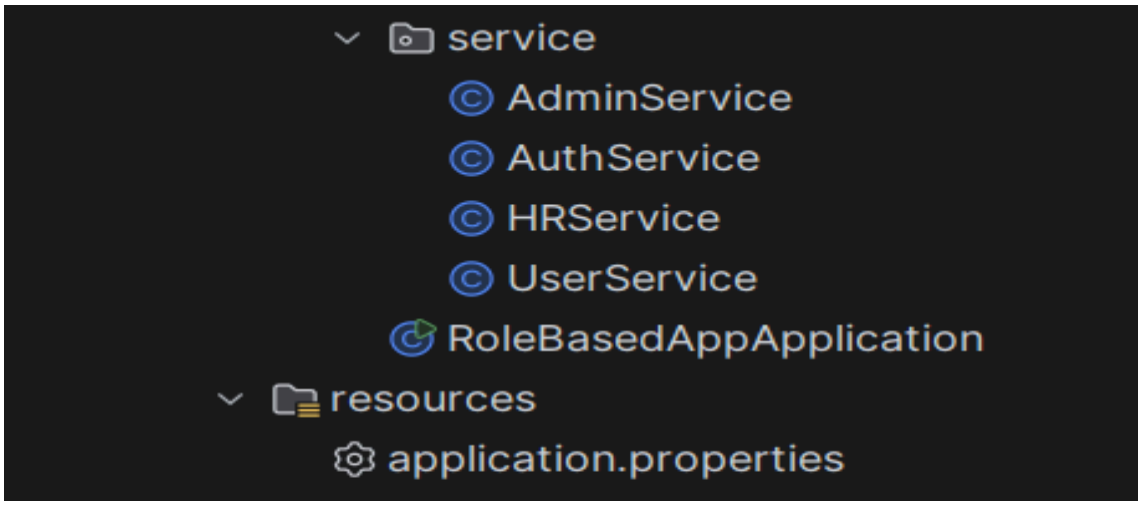
Step 1: Create Spring Boot Project in IntelliJ IDEA

1. **Open IntelliJ IDEA**
2. Click **"New Project"**
3. Select **"Spring Boot"** from the left sidebar
4. Configure the project:
    - **Name:** `RoleBasedApp`
    - **Language:** Java
    - **Type:** Maven
    - **Group:** `com.example`
    - **Artifact:** `rolebasedapp`
    - **Package name:** `com.example.rolebasedapp`

Role-Based Access Matrix

Role	Signup	Login	Access Own Data	View All Users	Delete Users	Update Users
USER	✅	✅	✅	❌	❌	❌
HR	❌	✅	✅	✅	❌	❌
ADMIN	❌	✅	✅	✅	✅	✅


java
com.example.rolebasedapp
config
SecurityConfig
controller
AdminController
AuthController
HRController
UserController
dto
AuthResponse
LoginRequest
SignupRequest
UserDTO
exception
GlobalExceptionHandler
UnauthorizedException
model
Role
User
repository
UserRepository
security
CustomUserDetailsService
JwtAuthenticationFilter
JwtUtil



Dependencies (pom.xml)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.2.0</version>
       <relativePath/>
   </parent>

   <groupId>com.app</groupId>
   <artifactId>role-based-app</artifactId>
   <version>1.0.0</version>
   <name>RoleBasedApp</name>
   <description>Advanced Role-Based Routing Application</description>

   <properties>
       <java.version>17</java.version>
   </properties>

   <dependencies>
       <!-- Spring Boot Web -->
       <dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-web</artifactId>
       </dependency>

       <!-- Spring Security -->
       <dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-security</artifactId>
```

```
    </dependency>

    <!-- MongoDB -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>

    <!-- JWT -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.5</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>


    <!-- Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Lombok -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <!-- Dev Tools -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
</dependencies>
```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

application.properties

```
# Server Configuration
server.port=8080
spring.application.name=role-based-app

# MongoDB Configuration
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=rolebasedapp

# JWT Configuration
jwt.secret=mySecretKeyForJWTTokenGenerationPleaseChangeThisInProduction1234567890
jwt.expiration=86400000

# Logging
logging.level.org.springframework.security=DEBUG
logging.level.com.example.rolebasedapp=DEBUG

# CORS
cors.allowed-origins=http://localhost:5173,http://localhost:3000
```

model/Role

```
package com.example.rolebasedapp.model;

public enum Role {
    USER,
    HR,
    ADMIN
```

```
}
```

model/User

```java
package com.example.rolebasedapp.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.index.Indexed;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "users")
public class User {

   @Id
   private String id;

   private String name;

   @Indexed(unique = true)
   private String email;

   private String password;

   private Role role;

   private LocalDateTime createdAt;

   private LocalDateTime updatedAt;

   public User(String name, String email, String password, Role role) {
       this.name = name;
       this.email = email;
       this.password = password;
       this.role = role;
       this.createdAt = LocalDateTime.now();
       this.updatedAt = LocalDateTime.now();
   }
}
```

repository/UserRepository

```java
package com.example.rolebasedapp.repository;

import com.example.rolebasedapp.model.Role;
import com.example.rolebasedapp.model.User;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends MongoRepository<User, String> {

   Optional<User> findByEmail(String email);

   boolean existsByEmail(String email);

   List<User> findByRole(Role role);

   long countByRole(Role role);
}
```

dto/AuthResponse

```java
package com.example.rolebasedapp.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AuthResponse {
   private String token;
   private String email;
   private String name;
   private String role;
   private String message;

   public AuthResponse(String token, String email, String name, String role) {
       this.token = token;
       this.email = email;
       this.name = name;
       this.role = role;
```

```
    }
}
```

dto/LoginRequest

```java
package com.example.rolebasedapp.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class LoginRequest {

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required")
    private String password;
}
```

dto/SignupRequest

```java
package com.example.rolebasedapp.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class SignupRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")
```

```
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;
}
```

dto/AuthResponse

```
package com.example.rolebasedapp.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AuthResponse {
    private String token;
    private String email;
    private String name;
    private String role;
    private String message;

    public AuthResponse(String token, String email, String name, String role) {
        this.token = token;
        this.email = email;
        this.name = name;
        this.role = role;
    }
}
```

dto/UserDTO

```
package com.example.rolebasedapp.dto;

import com.example.rolebasedapp.model.Role;
import com.example.rolebasedapp.model.User;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```java
import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserDTO {
   private String id;
   private String name;
   private String email;
   private String role;
   private LocalDateTime createdAt;

   public static UserDTO fromUser(User user) {
       return new UserDTO(
               user.getId(),
               user.getName(),
               user.getEmail(),
               user.getRole().name(),
               user.getCreatedAt()
       );
   }
}
```

security/CustomUserDetailsService

```java
package com.example.rolebasedapp.security;

import com.example.rolebasedapp.model.User;
import com.example.rolebasedapp.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

   private final UserRepository userRepository;

   @Override
   public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
       User user = userRepository.findByEmail(email)
```

```
                .orElseThrow(() -> new UsernameNotFoundException("User not found with
email: " + email));

        return new org.springframework.security.core.userdetails.User(
                user.getEmail(),
                user.getPassword(),
                Collections.singletonList(new SimpleGrantedAuthority("ROLE_" +
user.getRole().name()))
        );
    }
}
```

security/JwtAuthenticationFilter

```
package com.example.rolebasedapp.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException,
IOException {

        final String authorizationHeader = request.getHeader("Authorization");

        String username = null;
```

```
        String jwt = null;

        if (authorizationHeader != null && authorizationHeader.startsWith("Bearer "))
{
            jwt = authorizationHeader.substring(7);
            try {
                username = jwtUtil.extractUsername(jwt);
            } catch (Exception e) {
                logger.error("JWT Token extraction failed: " + e.getMessage());
            }
        }

        if (username != null && SecurityContextHolder.getContext().getAuthentication()
== null) {
            UserDetails userDetails =
this.userDetailsService.loadUserByUsername(username);

            if (jwtUtil.validateToken(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authenticationToken =
                        new UsernamePasswordAuthenticationToken(
                                userDetails,
                                null,
                                userDetails.getAuthorities()
                        );
                authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authenticationToken);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

security/JwtUtil

```
package com.example.rolebasedapp.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
```

```java
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    private SecretKey getSigningKey() {
        byte[] keyBytes = secret.getBytes(StandardCharsets.UTF_8);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public String extractRole(String token) {
        return extractClaim(token, claims -> claims.get("role", String.class));
    }

    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
                .setSigningKey(getSigningKey())
                .build()
                .parseClaimsJws(token)
                .getBody();
    }

    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    public String generateToken(String username, String role) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
```

```
        return createToken(claims, username);
    }

    private String createToken(Map<String, Object> claims, String subject) {
        return Jwts.builder()
                .setClaims(claims)
                .setSubject(subject)
                .setIssuedAt(new Date(System.currentTimeMillis()))
                .setExpiration(new Date(System.currentTimeMillis() + expiration))
                .signWith(getSigningKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
    }
}
```

config/SecurityConfig

```
package com.example.rolebasedapp.config;

import com.example.rolebasedapp.security.CustomUserDetailsService;
import com.example.rolebasedapp.security.JwtAuthenticationFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.authentication.configuration.Authentic
ationConfiguration;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecur
ity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
```

```java
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
@RequiredArgsConstructor
public class SecurityConfig {

   private final JwtAuthenticationFilter jwtAuthFilter;
   private final CustomUserDetailsService userDetailsService;

   @Value("${cors.allowed-origins}")
   private String allowedOrigins;

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
       http
               .csrf(csrf -> csrf.disable())
               .cors(cors -> cors.configurationSource(corsConfigurationSource()))
               .authorizeHttpRequests(auth -> auth
                       .requestMatchers("/api/auth/**").permitAll()
                       .requestMatchers("/api/admin/**").hasRole("ADMIN")
                       .requestMatchers("/api/hr/**").hasRole("HR")
                       .requestMatchers("/api/user/**").hasRole("USER")
                       .anyRequest().authenticated()
               )
               .sessionManagement(session -> session
                       .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
               )
               .authenticationProvider(authenticationProvider())
               .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

       return http.build();
   }

   @Bean
   public CorsConfigurationSource corsConfigurationSource() {
       CorsConfiguration configuration = new CorsConfiguration();
       configuration.setAllowedOrigins(Arrays.asList(allowedOrigins.split(",")));
       configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
       configuration.setAllowedHeaders(Arrays.asList("*"));
       configuration.setAllowCredentials(true);

       UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
```

```
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

exception/UnauthorizedException

```
package com.example.rolebasedapp.exception;

public class UnauthorizedException extends RuntimeException {
    public UnauthorizedException(String message) {
        super(message);
    }
}
```

exception/GlobalExceptionHandler

```
package com.example.rolebasedapp.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
```

```java
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<Map<String, String>> handleValidationExceptions(
           MethodArgumentNotValidException ex) {
       Map<String, String> errors = new HashMap<>();
       ex.getBindingResult().getAllErrors().forEach((error) -> {
           String fieldName = ((FieldError) error).getField();
           String errorMessage = error.getDefaultMessage();
           errors.put(fieldName, errorMessage);
       });
       return ResponseEntity.badRequest().body(errors);
   }

   @ExceptionHandler(BadCredentialsException.class)
   public ResponseEntity<Map<String, String>>
handleBadCredentials(BadCredentialsException ex) {
       Map<String, String> error = new HashMap<>();
       error.put("error", "Invalid email or password");
       return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
   }

   @ExceptionHandler(AccessDeniedException.class)
   public ResponseEntity<Map<String, String>>
handleAccessDenied(AccessDeniedException ex) {
       Map<String, String> error = new HashMap<>();
       error.put("error", "Access denied. You don't have permission to access this
resource.");
       return ResponseEntity.status(HttpStatus.FORBIDDEN).body(error);
   }

   @ExceptionHandler(UnauthorizedException.class)
   public ResponseEntity<Map<String, String>>
handleUnauthorized(UnauthorizedException ex) {
       Map<String, String> error = new HashMap<>();
       error.put("error", ex.getMessage());
       return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
   }

   @ExceptionHandler(IllegalArgumentException.class)
   public ResponseEntity<Map<String, String>>
handleIllegalArgument(IllegalArgumentException ex) {
       Map<String, String> error = new HashMap<>();
       error.put("error", ex.getMessage());
       return ResponseEntity.badRequest().body(error);
   }

   @ExceptionHandler(Exception.class)
   public ResponseEntity<Map<String, String>> handleGenericException(Exception ex) {
```

```
        Map<String, String> error = new HashMap<>();
        error.put("error", "An unexpected error occurred: " + ex.getMessage());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

service/AuthService

```
package com.example.rolebasedapp.service;

import com.example.rolebasedapp.dto.AuthResponse;
import com.example.rolebasedapp.dto.LoginRequest;
import com.example.rolebasedapp.dto.SignupRequest;
import com.example.rolebasedapp.model.Role;
import com.example.rolebasedapp.model.User;
import com.example.rolebasedapp.repository.UserRepository;
import com.example.rolebasedapp.security.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;
    private final AuthenticationManager authenticationManager;

    public AuthResponse signup(SignupRequest request) {
        // Check if user already exists
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new IllegalArgumentException("Email already registered");
        }

        // Create new user with USER role only
        User user = new User(
                request.getName(),
                request.getEmail(),
                passwordEncoder.encode(request.getPassword()),
                Role.USER
        );
```

```java
        userRepository.save(user);

        // Generate JWT token
        String token = jwtUtil.generateToken(user.getEmail(), user.getRole().name());

        return new AuthResponse(
                token,
                user.getEmail(),
                user.getName(),
                user.getRole().name(),
                "User registered successfully"
        );
    }

    public AuthResponse login(LoginRequest request) {
        // Authenticate user
        Authentication authentication = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                        request.getEmail(),
                        request.getPassword()
                )
        );

        // Get user details
        User user = userRepository.findByEmail(request.getEmail())
                .orElseThrow(() -> new IllegalArgumentException("User not found"));

        // Generate JWT token
        String token = jwtUtil.generateToken(user.getEmail(), user.getRole().name());

        return new AuthResponse(
                token,
                user.getEmail(),
                user.getName(),
                user.getRole().name(),
                "Login successful"
        );
    }
}
```

service/AdminService

```java
package com.example.rolebasedapp.service;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.model.Role;
import com.example.rolebasedapp.model.User;
import com.example.rolebasedapp.repository.UserRepository;
```

```java
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class AdminService {

   private final UserRepository userRepository;

   public List<UserDTO> getAllUsers() {
       return userRepository.findAll().stream()
               .map(UserDTO::fromUser)
               .collect(Collectors.toList());
   }

   public void deleteUser(String userId) {
       User user = userRepository.findById(userId)
               .orElseThrow(() -> new IllegalArgumentException("User not found"));

       // Prevent admin from deleting themselves or other admins
       if (user.getRole() == Role.ADMIN) {
           throw new IllegalArgumentException("Cannot delete admin users");
       }

       userRepository.deleteById(userId);
   }

   public UserDTO updateUser(String userId, UserDTO userDTO) {
       User user = userRepository.findById(userId)
               .orElseThrow(() -> new IllegalArgumentException("User not found"));

       // Prevent updating admin users
       if (user.getRole() == Role.ADMIN) {
           throw new IllegalArgumentException("Cannot update admin users");
       }

       // Update allowed fields
       if (userDTO.getName() != null && !userDTO.getName().isEmpty()) {
           user.setName(userDTO.getName());
       }

       if (userDTO.getEmail() != null && !userDTO.getEmail().isEmpty()) {
           // Check if email is already taken by another user
           if (userRepository.existsByEmail(userDTO.getEmail()) &&
                   !user.getEmail().equals(userDTO.getEmail())) {
               throw new IllegalArgumentException("Email already in use");
           }
           user.setEmail(userDTO.getEmail());
```

```
        }

        user.setUpdatedAt(LocalDateTime.now());
        User updatedUser = userRepository.save(user);

        return UserDTO.fromUser(updatedUser);
    }

    public long getTotalUserCount() {
        return userRepository.count();
    }
}
```

service/HRService

```
package com.example.rolebasedapp.service;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.model.Role;
import com.example.rolebasedapp.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class HRService {

    private final UserRepository userRepository;

    public List<UserDTO> getAllUsers() {
        return userRepository.findAll().stream()
                .map(UserDTO::fromUser)
                .collect(Collectors.toList());
    }

    public long getTotalUserCount() {
        return userRepository.count();
    }

    public long getUserCountByRole(String role) {
        try {
            Role roleEnum = Role.valueOf(role.toUpperCase());
            return userRepository.countByRole(roleEnum);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Invalid role: " + role);
```

```
        }
    }
}
```

service/UserService

```java
package com.example.rolebasedapp.service;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.model.User;
import com.example.rolebasedapp.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class UserService {

   private final UserRepository userRepository;

   public UserDTO getCurrentUserProfile() {
       String email =
SecurityContextHolder.getContext().getAuthentication().getName();
       User user = userRepository.findByEmail(email)
               .orElseThrow(() -> new IllegalArgumentException("User not found"));

       return UserDTO.fromUser(user);
   }
}
```

controller/AuthController

```java
package com.example.rolebasedapp.controller;

import com.example.rolebasedapp.dto.AuthResponse;
import com.example.rolebasedapp.dto.LoginRequest;
import com.example.rolebasedapp.dto.SignupRequest;
import com.example.rolebasedapp.service.AuthService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
```

```java
@RequiredArgsConstructor
public class AuthController {

   private final AuthService authService;

   @PostMapping("/signup")
   public ResponseEntity<AuthResponse> signup(@Valid @RequestBody SignupRequest
request) {
       AuthResponse response = authService.signup(request);
       return ResponseEntity.ok(response);
   }

   @PostMapping("/login")
   public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest
request) {
       AuthResponse response = authService.login(request);
       return ResponseEntity.ok(response);
   }
}
```

controller/AdminController

```java
package com.example.rolebasedapp.controller;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.service.AdminService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/admin")
@RequiredArgsConstructor
public class AdminController {

   private final AdminService adminService;

   @GetMapping("/users")
   @PreAuthorize("hasRole('ADMIN')")
   public ResponseEntity<List<UserDTO>> getAllUsers() {
       List<UserDTO> users = adminService.getAllUsers();
       return ResponseEntity.ok(users);
   }

   @DeleteMapping("/users/{userId}")
```

```java
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Map<String, String>> deleteUser(@PathVariable String userId)
{
        adminService.deleteUser(userId);
        Map<String, String> response = new HashMap<>();
        response.put("message", "User deleted successfully");
        return ResponseEntity.ok(response);
    }

    @PutMapping("/users/{userId}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<UserDTO> updateUser(
            @PathVariable String userId,
            @RequestBody UserDTO userDTO) {
        UserDTO updatedUser = adminService.updateUser(userId, userDTO);
        return ResponseEntity.ok(updatedUser);
    }

    @GetMapping("/stats")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Map<String, Object>> getUserStats() {
        Map<String, Object> stats = new HashMap<>();
        stats.put("totalUsers", adminService.getTotalUserCount());
        return ResponseEntity.ok(stats);
    }
}
```

controller/HRController

```java
package com.example.rolebasedapp.controller;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.service.HRService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/hr")
@RequiredArgsConstructor
public class HRController {

    private final HRService hrService;

    @GetMapping("/users")
```

```java
    @PreAuthorize("hasRole('HR')")
    public ResponseEntity<List<UserDTO>> getAllUsers() {
        List<UserDTO> users = hrService.getAllUsers();
        return ResponseEntity.ok(users);
    }

    @GetMapping("/stats")
    @PreAuthorize("hasRole('HR')")
    public ResponseEntity<Map<String, Object>> getUserStats() {
        Map<String, Object> stats = new HashMap<>();
        stats.put("totalUsers", hrService.getTotalUserCount());
        stats.put("regularUsers", hrService.getUserCountByRole("USER"));
        stats.put("hrUsers", hrService.getUserCountByRole("HR"));
        stats.put("adminUsers", hrService.getUserCountByRole("ADMIN"));
        return ResponseEntity.ok(stats);
    }
}
```

controller/UserController

```java
package com.example.rolebasedapp.controller;

import com.example.rolebasedapp.dto.UserDTO;
import com.example.rolebasedapp.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/user")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping("/profile")
    @PreAuthorize("hasRole('USER')")
    public ResponseEntity<UserDTO> getCurrentUserProfile() {
        UserDTO userDTO = userService.getCurrentUserProfile();
        return ResponseEntity.ok(userDTO);
    }
}
```

RoleBasedAppApplication

```java
package com.example.rolebasedapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RoleBasedAppApplication {

   public static void main(String[] args) {
       SpringApplication.run(RoleBasedAppApplication.class, args);
   }
}
```

Add the credentials for Admin and HR

Open MongoDB shell:

1 Use rolebsaedapp;

2

```
db.users.insertMany([
  {
    name: "HR Manager",
    email: "hr@company.com",
    password: "$2a$10$XYZ...", //add the bcrypt password
    role: "HR",
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    name: "System Admin",
    email: "admin@company.com",
```

```
    password: "$2a$10$ABC...",  //add the bcrypt password

    role: "ADMIN",
    createdAt: new Date(),
    updatedAt: new Date()
  }
]);
```